Srujana Duggineni – C00313483

Algorithm: ASL Classification Using CNNs (Exploring CNN architectures)

[American Sign Language Dataset]: Dataset

Reference Notebook: Notebook

Task	Actions Taken	Rationale	Difficulty Level (1-10)	Time Taken
Data Understanding	Explored the ASL dataset, inspected class distributions, and noted potential imbalances.	To understand dataset structure and ensure suitability for classification analysis.	3	20 min
Data Preprocessing	Normalized images (1./255), included data augmentation (flipping, rotation, brightness adjustment), and split dataset into training (80%) and validation (20%).	Ensures equal pixel values, improves generalization, and prevents overfitting.	5	30 min
Implemented Baseline Model - CNN	Used a custom CNN consisting of three convolutional layers, max pooling, and fully connected layers.	Used as a baseline to compare the transfer learning models.	7	1.5 hours
Transfer Learning - ResNet50	Utilized a pre-trained ResNet50 model (weights from ImageNet) and further fine-tuned it for the classification of ASL.	employs pre-trained features to make it more accurate and efficient.	8	1.5 hours
Transfer Learning - VGG16	Loaded pre-trained VGG16 with ImageNet weights, added classification layers, and trained on ASL dataset.	Analyzes another well-known CNN architecture as a baseline.	7	50 min

Training & Early Stopping	Trained the models with Adam optimizer and sparse categorical cross-entropy loss and applied early stopping.	Protects against overfitting while ensuring convergence	5	30min
Model Evaluation	Compared models based on accuracy and loss, inspected confusion matrices, and monitored validation performance.	Evaluates generalization capability and identifies misclassifications.	5	25 min
Observations & Analysis	Compared ResNet50, Simple CNN, and VGG16 results and noted ResNet50's high accuracy (95%) and VGG16's poor performance (60%).	Informs model selection and deployment alternatives.	4	30min
Conclusion & Analysis	Reported results, suggested VGG16's enhancements, and assessed deployment options.	Guides further optimization and real-world use.	3	15 min

1. Business Understanding

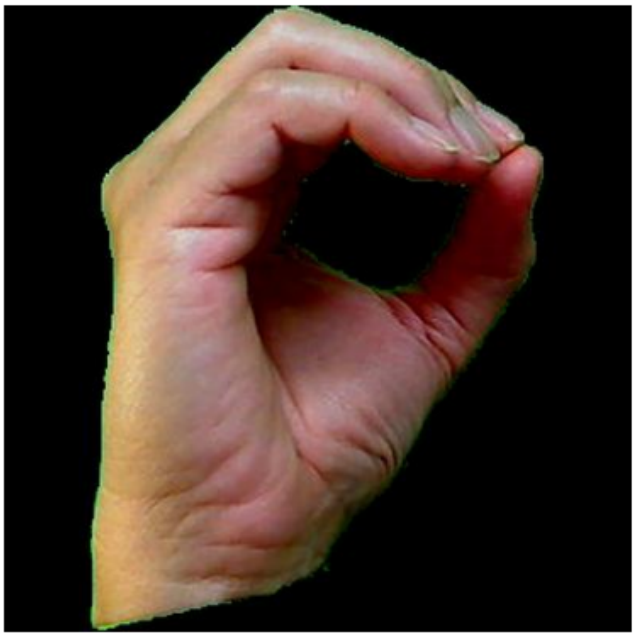
Objective: The objective of this work is to classify ASL hand signs using deep learning architectures like a user-specified Convolutional Neural Network (CNN), ResNet50, and VGG16.

The dataset consists of images representing different ASL signs, which are preprocessed and fed into deep learning architectures for classification. The models are trained and tested to determine the most accurate architecture in terms of accuracy and loss measures. Transfer learning is applied using ResNet50 and VGG16 for improved classification accuracy. Comparison of the two models and their pros and cons in ASL recognition is what this project aims.

2. Data Understanding:

Dataset is made up of images that depict ASL signs, grouped into folders representing different classes.

Sample from Datasets depicting Alphabet ‘0’



Samples from Datasets depicting Numerals ‘5’

Sample from Datasets depicting Alphabet ‘z’

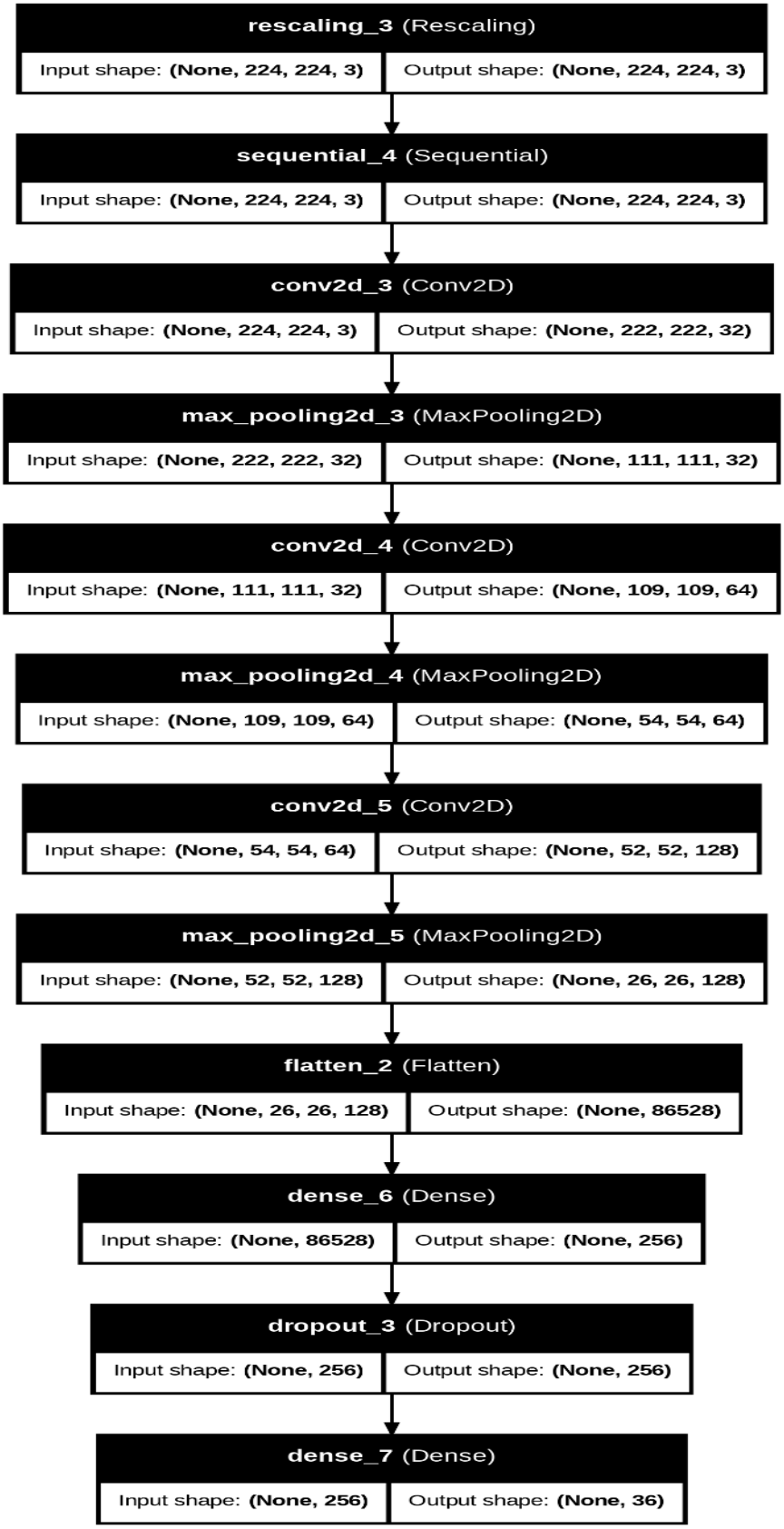
3. Data Preparation:

The data is loaded using TensorFlow's image_dataset_from_directory and resized to (224, 224) and batched for training. A sample of images is randomly plotted to examine differences in gesture representation such that different classes are distinguishable. The dataset is examined to confirm class distribution and image quality and to identify any potential imbalances requiring augmentation or oversampling. Furthermore, the structure of the image batch is examined to ensure correct loading of the dataset, e.g., samples per batch and label encoding.

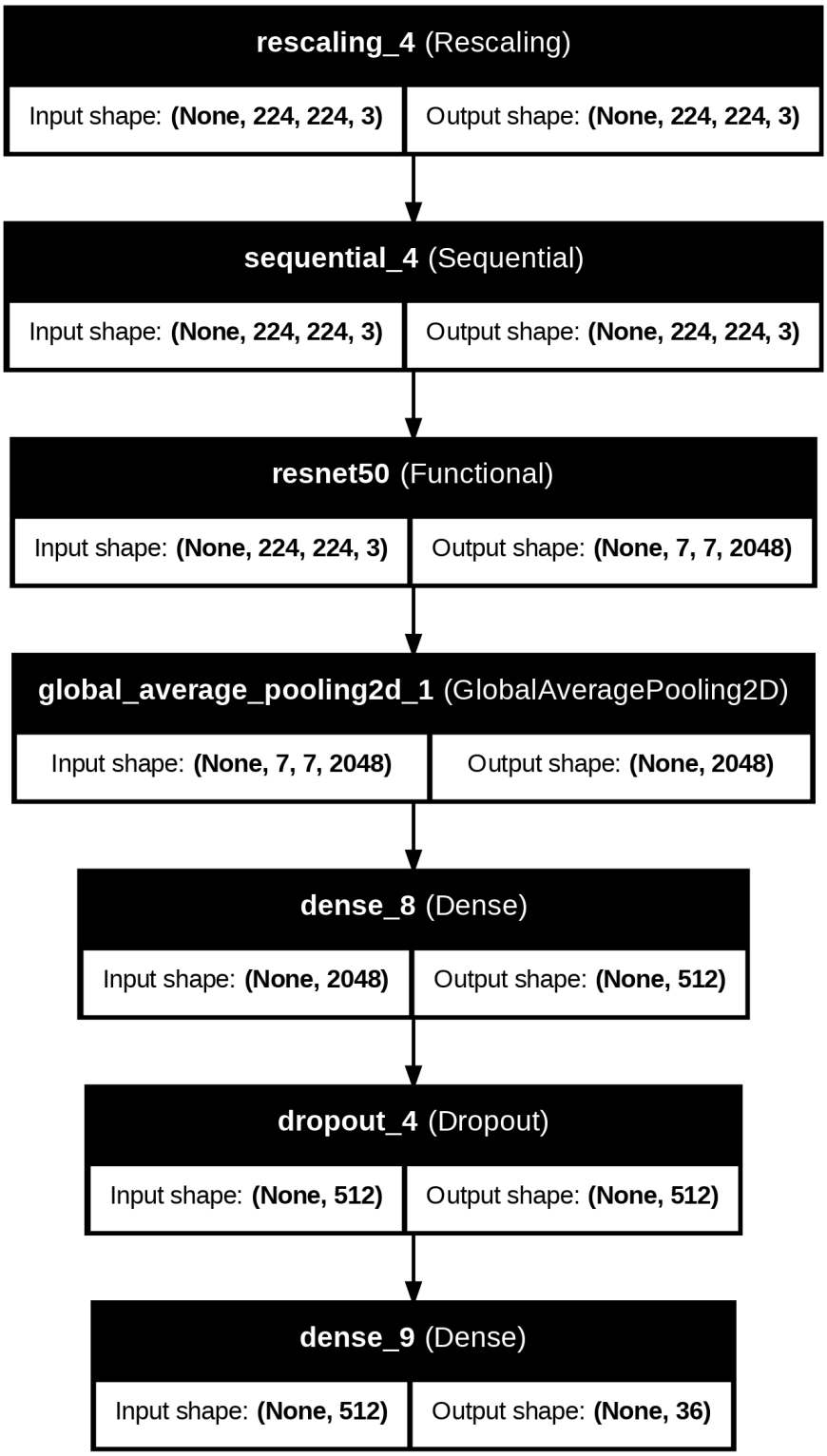
To prepare the dataset for model training, some preprocessing steps are carried out. Rescaling (1./255) is used to rescale the images, keeping pixel values in a standard range for deep learning models. Random flipping, rotation, and brightness changes are used as data augmentation techniques to enhance the generalization and robustness of the model. The dataset is split into training (80%) and validation (20%) sets to enable efficient model learning and evaluation. Processed data is now ready to be fed to CNN-based architectures for classification.

4. Modeling -Baseline Implementation (custom):

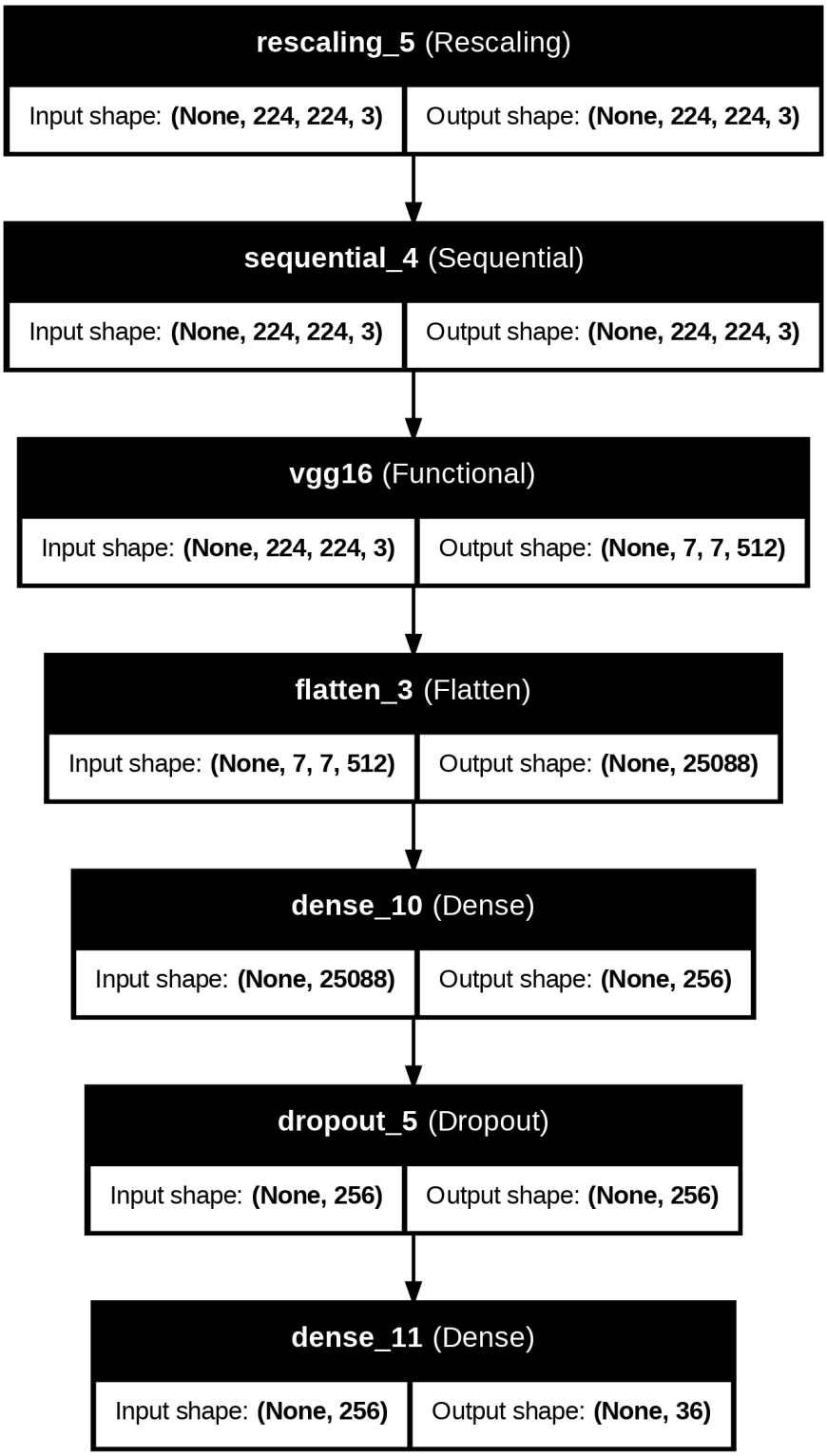
Three deep learning models are used for ASL classification. The first is a special CNN composed of three convolutional layers followed by max pooling and fully connected layers. This is a baseline model to define minimum classification capacity.


rescaling_3 (Rescaling)
Input shape: (None, 224, 224, 3)
Output shape: (None, 224, 224, 3)
sequential_4 (Sequential)
Input shape: (None, 224, 224, 3)
Output shape: (None, 224, 224, 3)
conv2d_3 (Conv2D)
Input shape: (None, 224, 224, 3)
Output shape: (None, 222, 222, 32)
max_pooling2d_3 (MaxPooling2D)
Input shape: (None, 222, 222, 32)
Output shape: (None, 111, 111, 32)
conv2d_4 (Conv2D)
Input shape: (None, 111, 111, 32)
Output shape: (None, 109, 109, 64)
max_pooling2d_4 (MaxPooling2D)
Input shape: (None, 109, 109, 64)
Output shape: (None, 54, 54, 64)
conv2d_5 (Conv2D)
Input shape: (None, 54, 54, 64)
Output shape: (None, 52, 52, 128)
max_pooling2d_5 (MaxPooling2D)
Input shape: (None, 52, 52, 128)
Output shape: (None, 26, 26, 128)
flatten_2 (Flatten)
Input shape: (None, 26, 26, 128)
Output shape: (None, 86528)
dense_6 (Dense)
Input shape: (None, 86528)
Output shape: (None, 256)
dropout_3 (Dropout)
Input shape: (None, 256)
Output shape: (None, 256)
dense_7 (Dense)
Input shape: (None, 256)
Output shape: (None, 36)

The second model, ResNet50, is a pre-trained convolutional neural network with ImageNet weights, and the subsequent layers are fine-tuned to adapt to ASL classification.



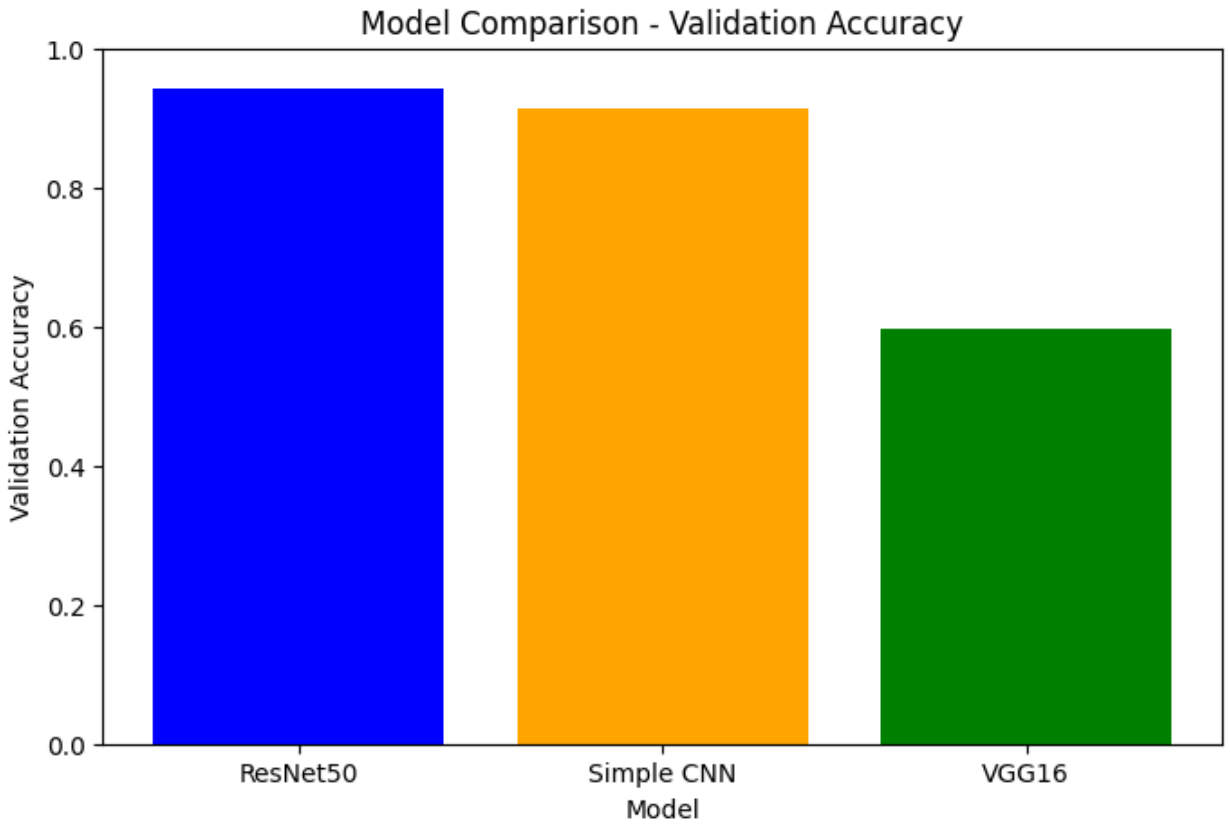
The third model, VGG16, is similarly a pre-trained network that is adapted by appending dense layers for classification.



Softmax activation is applied in all models for multi-class classification, and the models are compiled using the Adam optimizer and sparse categorical cross-entropy loss function. Training is carried out with early stopping to prevent overfitting, and the models are evaluated against the validation set to determine performance.

5. Evaluation:

The models are tested on standard performance measures like accuracy and loss. The training and validation curves are inspected to determine each model's capacity to generalize to new examples. Simple CNN, ResNet50, and VGG16 training histories are compared to inspect trends such as overfitting or underfitting. Confusion matrices can be graphed to explore the errors in classification in more detail. The effect of transfer learning is seen when we compare ResNet50 and VGG16 against the customized CNN on accuracy and learning approach. Based on validation accuracy and capacity to generalize the best-performing model is choosen



Observations:

- ResNet50: Exhibits the highest validation accuracy, close to 95%.
- Simple CNN: Achieves a slightly lower validation accuracy, around 92%.
- VGG16: Shows significantly lower validation accuracy, around 60%.

Summary:

- The Best Performing Model is ResNet50: Considering validation accuracy, ResNet50 performed well than Simple CNN and VGG16. This indicates that the ResNet50 model is an appropriate model for the provided task and dataset.
- Simple CNN is a Viable Alternative: Though this is not as accurate as ResNet50, Simple CNN, achieves high validation accuracy. If computational resources are limited this is a good approach because it is a bit less complex model than ResNet50.
- VGG16 struggling on the Task: VGG16 does significantly worse when compared to the other two models. This shows that VGG16 is not a suitable architecture for this given task or dataset.

Model Selection Implications: Given a model to deploy, ResNet50 would be the top choice due to its outstanding performance. But if model size or computational cost is a critical factor, Simple CNN would be an suffice trade-off.

Further work Investigation: The significant difference in performance of VGG16 compared to the other models needs further investigation. For VGG16, experimenting with different hyperparameters, data augmentation techniques, or transfer learning schemes could be beneficial to boost its performance.